CAMPUSLINK

UCS 503 • SOFTWARE ENGINEERING



CampusLink

“Find your people — beyond your friend circle.”

A goal-matching and resource-sharing platform for college campuses

SOFTWARE REQUIREMENTS SPECIFICATION

SUBMITTED BY

Govind	1024030968
Ashish	1024030096
Divya	1024030099
Divyansh	1024030064

Group 6

SUBMITTED TO

Anushka Ma’am

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Document control

Title	CampusLink — Software Requirements Specification
Course	UCS503 Software Engineering, Computer Science and Engineering Department
Institute	Thapar Institute of Engineering and Technology, Patiala
Project repository	`github.com/govinda16293/CampusLink`
Status	Prepared for evaluation. Build status is recorded per requirement (Chapter 3).
Authors	Govind (1024030968), Ashish (1024030096), Divya (1024030099), Divyansh (1024030064)
Supervisor	Anushka Ma'am

Project at a glance

15	66	16	4 of 13
use cases	numbered requirements	endpoints live	build steps complete

How to read the status markers

Implemented **Implemented.** Built, merged in the repository and covered by automated tests or a screen in the prototype.

Partial **Partial.** Part of the behaviour or its data model exists; the rest waits for a later build step.

Planned **Planned.** Specified and scheduled in the build order of Section 5.3; not yet in the repository.

Future scope **Future scope.** Considered in the proposal but not part of this project's build. Kept here as a possible extension.

> This specification keeps the *specified* product and the *built* product apart. Of the 66 requirements, 29 are implemented, 4 are partial, 31 are planned and 2 are future scope. Nothing marked Planned is claimed to exist in the prototype.

Contents

1 Introduction **1**
- 1.1 Purpose . . . 1
- 1.2 Project Scope . . . 1
- 1.3 System Work-flow Overview . . . 1
- 1.4 Definitions . . . 2

2 Overall Description **3**
- 2.1 Product Perspective . . . 3
- 2.2 User Classes and Characteristics . . . 3
- 2.3 Product Functions . . . 3
- 2.4 Operating Environment . . . 4
- 2.5 Design and Implementation Constraints . . . 4
- 2.6 Assumptions and Dependencies . . . 4
- 2.7 System Architecture . . . 4
- 2.8 Design Activities and Architectural Models . . . 4

3 System Features **9**
- 3.1 Description and Priority . . . 9
- 3.2 Functional Requirements (Module Breakdown) . . . 9
- 3.3 Business Rules Behind the Requirements . . . 15
- 3.4 External Interface Requirements . . . 15

4 Other Nonfunctional Requirements **17**
- 4.1 Performance Requirements . . . 17
- 4.2 Security Requirements . . . 17
- 4.3 Software Quality Attributes . . . 18
- 4.4 Business Rules and Evaluation Metrics . . . 18

5 Other Requirements **20**
- 5.1 Future System Evolution . . . 20
- 5.2 Implementation Roadmap . . . 20
- 5.3 Risks . . . 21

6 Assumptions and Open Points **21**

1 Introduction

1.1 Purpose

On a campus of several thousand students, everyday coordination still runs through batch WhatsApp groups, hostel groups and word of mouth: a gym partner, a shared cab to the airport, someone who has last semester's notes. That works only if you already know the right people. It also gives no safe structure for dealing with a stranger, and no record of who keeps flaking on plans.

CampusLink is a web platform that closes this gap. A student *posts a goal* (an activity that needs another person). The goal appears on a public campus feed. Interested students *request to join*. Once enough are in, a small *lobby* forms and the group takes it from there. A trust-and-safety layer gives the interaction enough structure to be safe with someone you do not know yet: anonymous posting, a gender-preference filter, a visible reliability score, report and block, and no-show tracking.

This document is the Software Requirements Specification for CampusLink. It states what the system shall do, the constraints it works under and the quality it must reach. For every requirement it also records whether the requirement is already built, partly built, planned or left as future scope, so the document can be read against the repository now and again at final submission.

1.2 Project Scope

CampusLink serves one closed community: students holding a verified `@thapar.edu` address. Within it, the product supports the loop *post, discover, request, approve, lobby, coordinate, close and rate*. The same loop is reused for a smaller second use case, academic resource sharing: a student posts that they need notes for a subject, and whoever has them answers in a private one-to-one thread instead of a public lobby.

The success of the product is judged by one primary metric, the *lobby fulfilment rate* (the share of posted goals that reach their required headcount in time), and by four secondary metrics defined in Section 4.4. They will be measured in a pilot with 15–20 students from the team's own batch.

In scope

- College-domain accounts with e-mail one-time password
- Goal posting, public feed, filters
- Request-to-join with manual or automatic approval
- Lobbies and real-time chat
- Notes and resource requests with private threads
- Anonymous posting, gender preference, report and block
- Reliability score, attendance confirmation, ratings
- Rule-based suggestions and priority ordering
- Evaluation metrics and free-tier deployment

1.3 System Work-flow Overview

Figure 1.1 shows the core loop in seven stages. The first two stages are built and tested. The trust-and-safety layer is not a stage of its own: it wraps every stage, because its job is to make each hand-off between strangers safer.

Every stage depends on the one before it. The poster's goal starts the cycle; the feed brings it to people who want the same thing; the request and approval step keeps the poster in control of who joins; the lobby turns accepted strangers into a group; and the closing step feeds back into the reliability score, which every later feed card displays.

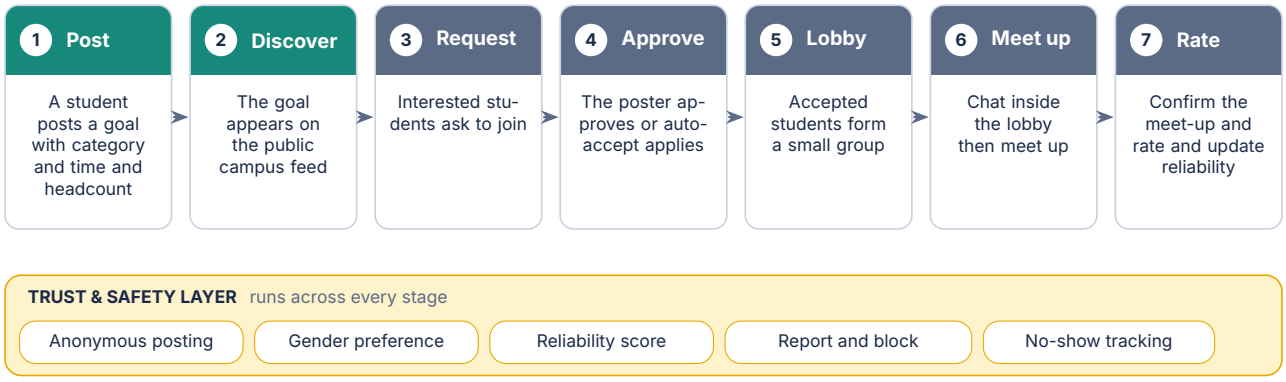


Figure 1.1. Entire system work-flow: the seven stages of the CampusLink loop and the trust layer that spans them

1.4 Definitions

Term	Meaning
Goal	An activity a student needs other people for: a category, a title, a description, an optional date and time and a headcount.
Poster	The student who posted a goal.
Requester	A student who asked to join someone else's goal.
Headcount	The number of *other* people the poster needs. The poster is not counted; a gym partner is a headcount of 1.
Feed	The public list of open goals, newest first, filterable by category and date.
Lobby	The group formed around a goal once its first request is accepted. Members chat inside it.
Anonymous goal	A goal whose poster's name and photo are hidden from the feed until a requester is accepted. The reliability score stays visible.
Gender preference	A per-goal option (`ANY`, `MALE_ONLY`, `FEMALE_ONLY`) restricting who may request to join.
Auto-accept	A per-goal option that accepts valid requests immediately until the headcount is met.
Reliability score	An integer, starting at 100, that summarises how dependable a student has been. Shown on every profile and feed card.
One-time password	A single-use six-digit code, e-mailed at sign-up and valid for 10 minutes.
JavaScript Object Notation Web Token	A signed session token carried in the `Authorization: Bearer` header after login.
Data transfer object	The JavaScript Object Notation shape that the application programming interface returns, built by a serializer from a database row.
Single-page application	The React front end.
Object-relational mapper	Prisma in this project.
Zod	TypeScript schema-validation library, used for request validation and shared between front end and back end.
scrypt	Memory-hard password-hashing function used for stored passwords.
Identifier prefixes	`UC-nn` is a use case, `FR-n.n` a functional requirement and `FEAT-nn` a feature.

2 Overall Description

2.1 Product Perspective

CampusLink is a new, self-contained product; it replaces nothing that exists in software, only informal chat groups. It is a 3-tier web application: a React single-page application in the browser, a stateless Express application programming interface, and a relational database reached only through the application programming interface. The only systems it talks to are an Simple Mail Transfer Protocol relay for the sign-up passcode and, later, an object store for notes attachments. Figure 2.1 draws that boundary, and Figure 2.2 opens the box.

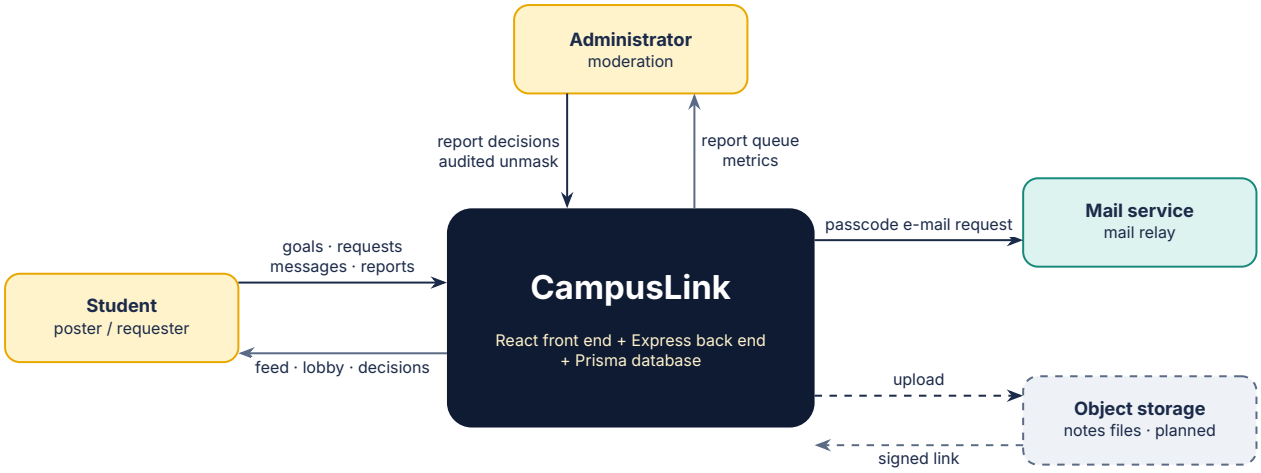


Figure 2.1. System context: CampusLink and the people and services around it

2.2 User Classes and Characteristics

The system distinguishes three user classes . Within a goal, one student account plays one of three roles at a time.

- **Visitor:** not signed in. Sees the landing page and can sign up, verify the e-mail one-time password and log in. Cannot read the feed.
- **Student:** a verified `@thapar.edu` account. Posts goals, browses the feed, requests to join, chats in lobbies, rates, reports and blocks. Comfortable with a phone browser; expects pages to load quickly.
- **Administrator:** a student account whose `role` is `ADMIN`. Reviews reports, may unmask an anonymous poster for a recorded reason, and reads the evaluation metrics.

2.3 Product Functions

The core product functions are:

1. **Verified, college-only accounts.** Sign-up restricted to the college domain and confirmed by an e-mailed one-time password.
2. **Goal posting and public feed.** Post an activity, filter the feed by category and date, and read a goal in detail.
3. **Request and approval.** Ask to join; the poster approves, rejects, or lets auto-accept decide.
4. **Lobby and real-time chat.** A group chat opens once a lobby forms.
5. **Notes and resource sharing.** Private one-to-one answers to a notes request, with file attachments.
6. **Trust and safety.** Anonymous posting, gender preference, report and block, and audited administrator unmasking.
7. **Reliability and feedback.** Attendance confirmation, no-show flags, ratings and a visible reliability score.

8. **Matching and metrics.** Rule-based suggestions, reliability-aware request ordering and the evaluation metrics of Section 4.4.

2.4 Operating Environment

CampusLink runs in any standard computing environment that has a modern web browser on the client side and Node.js on the server side.

Layer	Technology
Client	Chrome, Firefox, Safari, Edge and mobile browsers. React 18, Vite 6, Tailwind CSS v4, React Router 7, Zustand 5; plain JavaScript.
Server	Node.js and Express 5 in TypeScript; Zod 3 validation; signed session tokens; `scrypt` password hashing; nodemailer for Simple Mail Transfer Protocol; helmet and `express-rate-limit`.
Data	Prisma 5 object-relational mapper over SQLite in development and PostgreSQL in production.
Shared code	A workspace package of Zod schemas, enumerations and constants imported by both front end and back end.
Quality tooling	Vitest and Supertest for tests; ESLint, Prettier and the TypeScript compiler; GitHub Actions runs format, lint, type-check, test and build on every push.
Planned	Socket.IO for chat, Cloudinary or Firebase for files; Vercel for the front end, Render or Railway for the application programming interface.

Table 2.1. Operating environment and technology stack

2.5 Design and Implementation Constraints

- The platform shall be usable on an average student phone or laptop, with no heavy client-side processing.
- The deployment shall fit standard free-tier hosting, so that a live demonstration link exists.
- Free-tier hosts have an ephemeral file system. Avatars are therefore stored in the database until object storage arrives in Step 7.
- SQLite has no native enumerations or arrays. Enumerated columns are strings checked at the application programming interface boundary, and interests are stored as one comma-separated string.
- Identity is guaranteed only by the college e-mail domain; there is no other identity proof.

2.6 Assumptions and Dependencies

- Students have a working `@thapar.edu` mailbox and can receive e-mail within a few minutes.
- The Simple Mail Transfer Protocol relay is available. If it is not, sign-up cannot complete, because verification is the only way to activate an account.
- The pilot has 15 to 20 users, so the feed is small and no sharding or caching layer is needed.
- Each goal’s date and time are interpreted in the poster’s browser time zone and stored in Coordinated Universal Time.
- Verification happens once, at sign-up. Later logins use the password only.

2.7 System Architecture

Figure 2.2 shows the three tiers and the pipeline every request passes through. Business logic sits in services; a serializer is the only place that turns a database row into a response, which is why anonymity is enforced there and never in the browser. Dashed items are planned.

2.8 Design Activities and Architectural Models

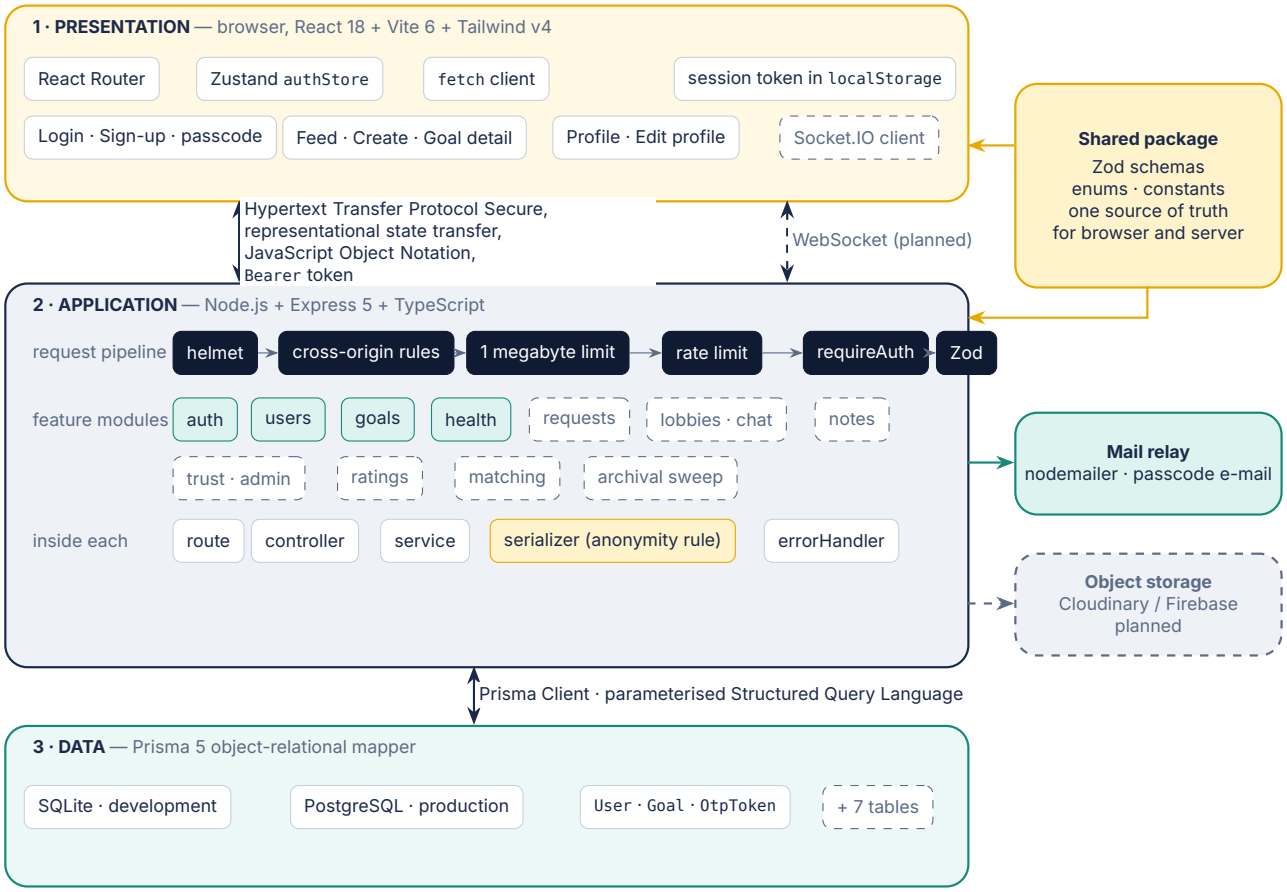


Figure 2.2. Three-tier architecture of CampusLink; dashed items are planned

2.8.1 Use-case Catalogue

The use cases specify the interactions between the actors and the system. The mail service delivers the one-time password for *Verify E-mail One-Time Password*; *Sign Up* and *Report or Block User* `include` the verification and review use cases, and *Coordinate in Lobby Chat* follows *Request to Join* through an `extend` relationship that applies only if the request is accepted. The catalogue in Table 2.2 gives each use case an identifier, which the requirement tables in Chapter 3 trace back to.

2.8.2 Relational Schema

Figure 2.3 shows the three tables that exist in the database today (`User`, `Goal`, `OtpToken`); seven more tables arrive with Steps 4 to 9.

The link between `OtpToken` and `User` is by e-mail address only and has no foreign key. That is deliberate: a passcode is issued before the account is verified, and a code row must be able to outlive nothing but its own expiry. Foreign keys with cascade delete are used elsewhere; deleting a student removes their goals.

2.8.3 Sequence Diagram: Sign-up and One-Time Password Verification

Figure 2.4 shows the sign-up path, which is built and tested. The application programming interface validates the college domain, stores a salted `scrypt` hash of the password and only a hash of the passcode, and e-mails the code. A wrong code increments an attempt counter and the code is burned after five misses, so a six-digit code cannot be brute-forced inside its ten-minute life.

2.8.4 Data-flow Diagrams

Figure 2.5 covers the three built processes; the remaining processes (requests, lobby chat, trust and safety, ratings) follow the same pattern and are planned. Each arrow is labelled F1, F2 and so on. Round boxes are processes, open rectangles are data stores and shaded boxes are external entities.

Identifier	Use case	Actor(s)	Status
UC-01	Sign Up	Student	Implemented
UC-02	Verify E-mail One-Time Password	Student, Mail Service	Implemented
UC-03	Log In	Student	Implemented
UC-04	Manage Profile	Student	Implemented
UC-05	Post a Goal	Student	Implemented
UC-06	Browse / Filter Feed	Student	Implemented
UC-07	View Goal Detail	Student	Implemented
UC-08	Cancel a Goal	Student	Implemented
UC-09	Request to Join	Student	Planned
UC-10	Approve / Reject Request	Student (poster)	Planned
UC-11	Coordinate in Lobby Chat	Student	Planned
UC-12	Share Notes / Resources	Student	Planned
UC-13	Rate Completion / Flag No-show	Student	Planned
UC-14	Report or Block User	Student	Planned
UC-15	Review Reports	Administrator	Planned

Table 2.2. Use-case catalogue (identifiers used throughout this document)

2.8.5 State-machine Models

The goal is the main entity with a life cycle (Figure 2.6). Solid transitions are implemented; dashed transitions are planned. Join requests move through `PENDING`, `ACCEPTED`, `REJECTED` or `WITHDRAWN`, and lobbies through `FORMING`, `CONFIRMED`, `COMPLETED` or `CANCELLED` (Steps 4 and 5).

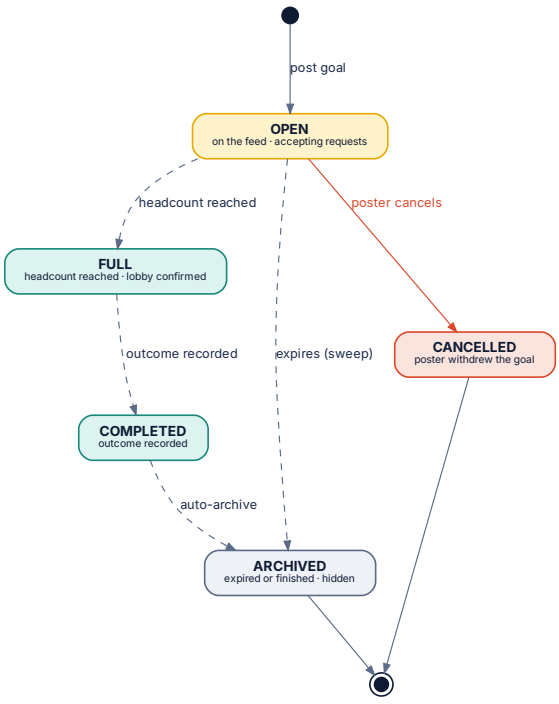


Figure 2.6. State machine of a goal



Figure 2.3. Relational schema, implemented part (`User`, `Goal`, `OtpToken`)

From	To	Trigger	Status
(start)	**OPEN**	The poster submits a valid goal.	Implemented
OPEN	**CANCELLED**	The poster cancels. Only an open goal can be cancelled; anyone else gets `403`.	Implemented
OPEN	**FULL**	The accepted count reaches the headcount.	Planned
FULL	**COMPLETED**	Outcomes of the meet-up are recorded.	Planned
OPEN	**ARCHIVED**	The goal expires and the sweep marks it. Until then expired goals are hidden at query time.	Planned
COMPLETED	**ARCHIVED**	Automatic archival after completion.	Planned

Table 2.3. Goal transitions

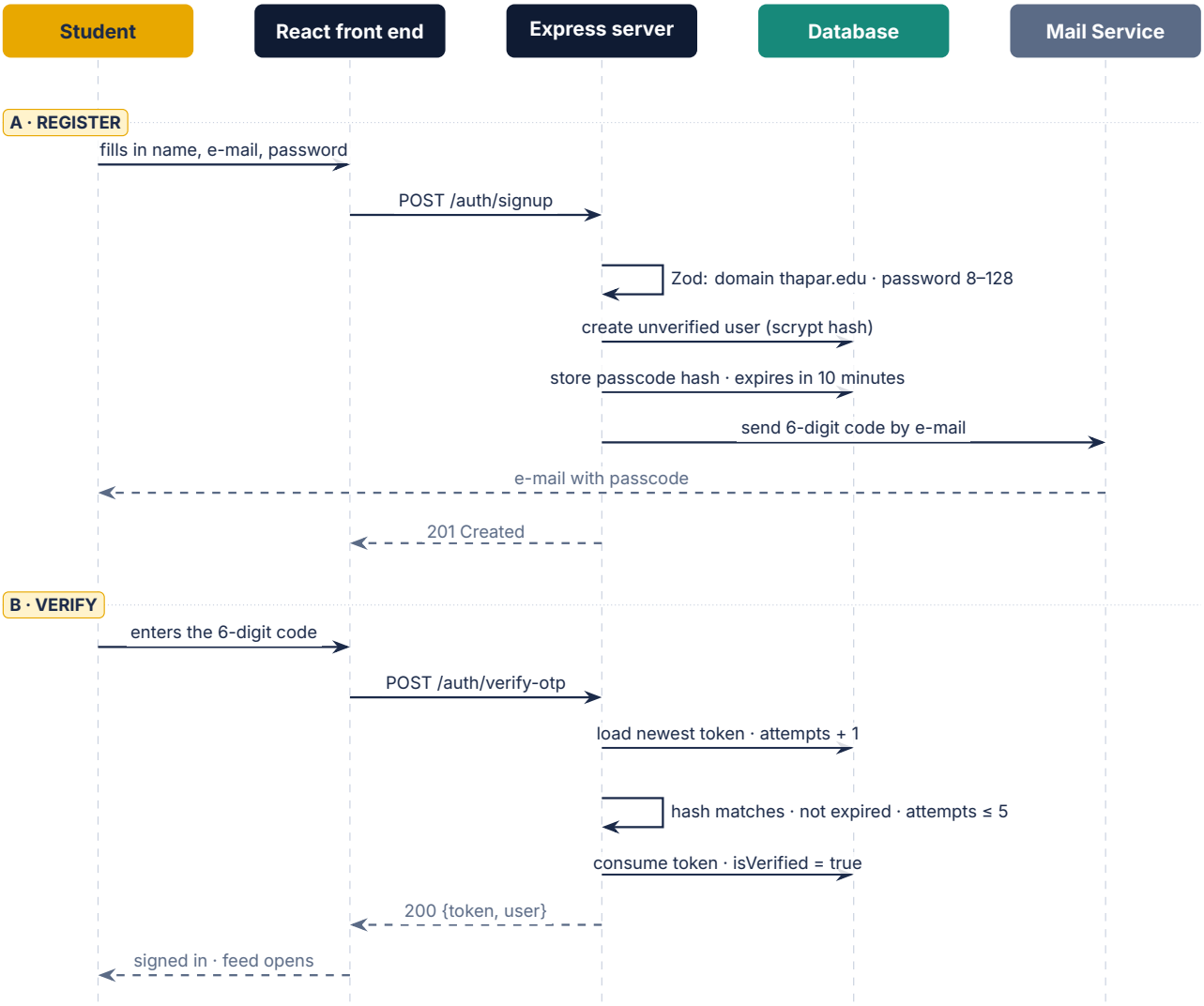


Figure 2.4. Sequence diagram: registration and e-mail one-time password verification

The enumeration also contains `CLOSED`. No transition into it is defined yet, so it is reserved and unused.

Identifier	Priority	Feature	Description	Module	Status

3 System Features

3.1 Description and Priority

Features are grouped into three priority tiers. P1 features form the core loop; without them the product does not work. P2 features make it trustworthy. P3 features improve discovery and measurement.

Identifier	Priority	Feature	Description	Module	Status
FEAT-01	High	**Verified student accounts**	Sign-up restricted to the college domain, e-mail one-time password, signed session tokens.	Module 1	Implemented
FEAT-02	High	**Goal posting and feed**	Post a goal; filter and page through the public feed; goal detail; cancel.	Module 3	Implemented
FEAT-03	High	**Join request and approval**	Request to join; manual approval or auto-accept; gender-preference enforcement.	Module 4	Planned
FEAT-04	High	**Lobby and real-time chat**	Group formed from accepted requests; Socket.IO chat with history.	Module 5	Planned
FEAT-05	High	**Trust and safety layer**	Anonymous posting, gender preference, report and block, audited unmasking.	Module 3, Module 7	Partial
FEAT-06	Medium	**Student profile**	Branch, year, interests, avatar, completeness, public view.	Module 2	Implemented
FEAT-07	Medium	**Reliability and ratings**	Attendance confirmation, no-show flags, ratings, reliability score.	Module 8	Partial
FEAT-08	Medium	**Notes and resource sharing**	Private one-to-one threads and file attachments for notes requests.	Module 6	Planned
FEAT-09	Low	**Matching and priority queue**	Suggested goals and reliability-aware ordering of requests.	Module 9	Planned
FEAT-10	Low	**Metrics, archival, deployment**	Evaluation metrics, archival sweep, continuous integration, free-tier deployment.	Module 10	Partial

3.2 Functional Requirements (Module Breakdown)

The functional requirements are split into ten modules that follow the build order. Each table lists the requirement, its priority and its status; use-case traceability is in the catalogue of Section 2.8. Figure 3.1 summarises the coverage.

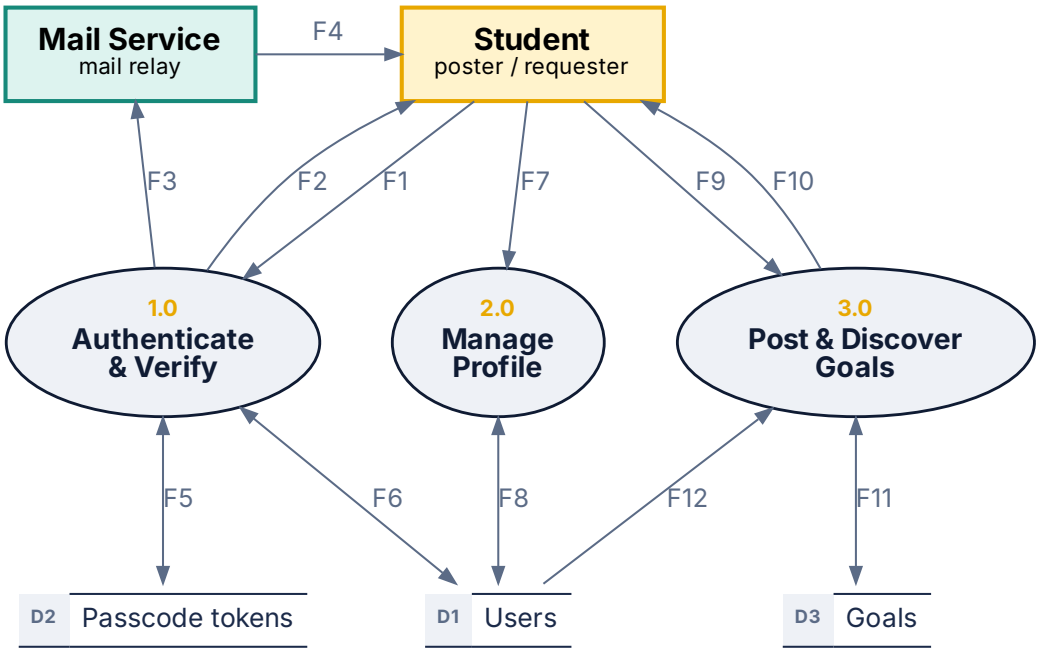


Figure 2.5. Data-flow diagram, implemented processes (1.0 to 3.0)

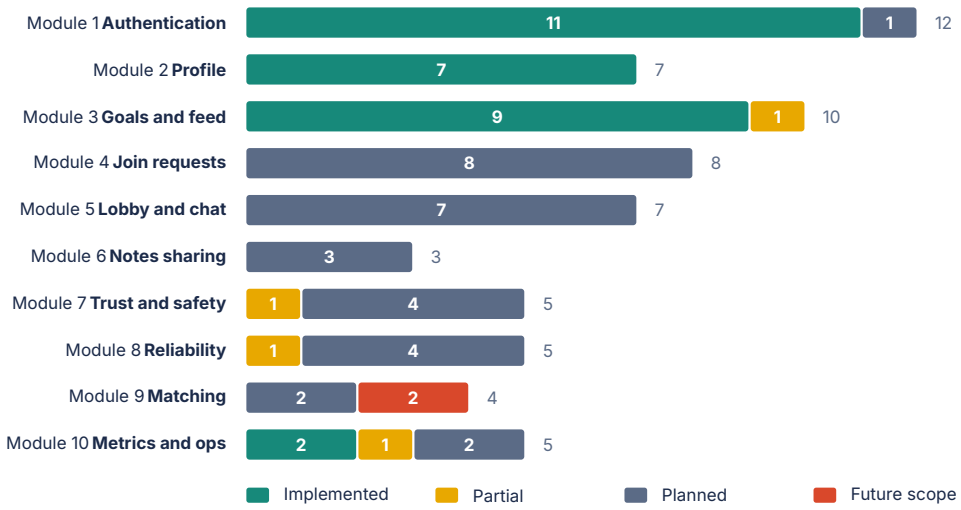


Figure 3.1. Requirements per module by build status (number of requirements at the right of each bar)

3.2.1 Module 1: Authentication and Account Verification

Build Step 1 — 11 of 12 implemented

College-domain sign-up, e-mail one-time password verification and session handling.

Identifier	Requirement	Priority	Status
FR-1.1	The system shall register a student from a name, an e-mail address and a password. The e-mail shall be stored lower-cased and its domain shall be `thapar.edu`; any other domain shall be rejected.	High	Implemented
FR-1.2	The password shall be 8 to 128 characters long and shall be stored only as a salted `scrypt` hash.	High	Implemented

Identifier	Requirement	Priority	Status
FR-1.3	On sign-up the system shall e-mail a 6-digit one-time password. Only a hash of the code shall be stored; the code shall expire after 10 minutes and be single-use.	High	Implemented
FR-1.4	A code shall be burned after 5 incorrect attempts, after which the student shall have to request a new one.	High	Implemented
FR-1.5	A student shall be able to request a new code. The response shall be identical for unknown and already-verified addresses, so that the endpoint cannot be used to find out who has signed up.	Medium	Implemented
FR-1.6	Successful verification shall mark the account as verified and return a session token together with the private user record.	High	Implemented
FR-1.7	Login with e-mail and password shall return a signed JavaScript Object Notation Web Token (default lifetime 7 days) and the private user record. The error for an unknown e-mail and for a wrong password shall be the same.	High	Implemented
FR-1.8	A student whose account is not yet verified shall not be able to log in and shall be told to complete verification.	High	Implemented
FR-1.9	Every route other than health, the auth endpoints and the avatar image shall require an `Authorization: Bearer` token and shall answer `401` when it is missing, malformed or expired.	High	Implemented
FR-1.10	The authentication endpoints shall be rate-limited to 20 requests per client per 15 minutes.	Medium	Implemented
FR-1.11	The client shall let a student log out by discarding the stored session token.	Low	Implemented
FR-1.12	A student who has forgotten the password shall be able to reset it through an e-mailed one-time password. The login page already shows a ”Forgot password?” link; no reset endpoint exists yet.	Medium	Planned

3.2.2 Module 2: Student Profile

Build Step 2 — 7 of 7 implemented

Profile fields, avatar, completeness and the public view of another student.

Identifier	Requirement	Priority	Status
FR-2.1	An authenticated student shall be able to read their own profile, including e-mail and a profile-completeness percentage.	High	Implemented
FR-2.2	A student shall be able to edit name (2 to 80 letters, spaces, hyphens, apostrophes and periods), branch, year of study (1 to 4) and gender (`MALE`, `FEMALE`, `OTHER`, `UNDISCLOSED`).	High	Implemented
FR-2.3	A student shall be able to keep up to 10 interest tags, each at most 24 characters and free of commas. Case-insensitive duplicates shall be dropped.	Medium	Implemented
FR-2.4	The profile update shall reject unknown fields. A student shall not be able to set their own role, verification status, reliability score or counters.	High	Implemented
FR-2.5	A student shall be able to upload and delete an avatar (Joint Photographic Experts Group, Portable Network Graphics or WebP, resized to 320 pixels, at most 250 kilobytes).	Low	Implemented
FR-2.6	Avatar bytes shall be served from their own cacheable endpoint and shall never be inlined into goal or user JavaScript Object Notation.	Low	Implemented
FR-2.7	A student shall be able to view another student’s public profile (name, branch, year, interests, reliability score, goals completed). The e-mail address and role shall never be included.	Medium	Implemented

3.2.3 Module 3: Goal Posting, Feed and Discovery

Build Step 3 9 of 10 implemented

Posting a goal, the campus feed, anonymity and cancellation.

Identifier	Requirement	Priority	Status
FR-3.1	A verified student shall be able to post a goal with a category (`SPORTS`, `GYM`, `STUDY`, `TRAVEL`, `FOOD`, `EVENTS`, `NOTES`, `OTHER`), a title of 4 to 80 characters, a description of 10 to 600 characters, a headcount of 1 to 20 (not counting the poster) and an optional date and time.	High	Implemented
FR-3.2	A goal shall carry three trust options: anonymous posting, a gender preference (`ANY`, `MALE_ONLY`, `FEMALE_ONLY`) and auto-accept.	High	Implemented
FR-3.3	A dated goal shall not be accepted for a time in the past (one minute of clock tolerance is allowed).	Medium	Implemented
FR-3.4	A dated goal shall expire at its date and time. An undated goal shall expire 48 hours after it is created.	High	Implemented
FR-3.5	The feed shall list only open, unexpired goals, newest first, with cursor pagination (default page size 20, maximum 50).	High	Implemented
FR-3.6	The feed shall be filterable by category and by date range, and shall offer a switch that shows only the viewer's own goals.	High	Implemented
FR-3.7	For an anonymous goal the server shall withhold the poster's name and photo from everyone except the poster. The poster's reliability score shall remain visible. The rule shall be applied in the server-side serializer, never in the user interface.	High	Implemented
FR-3.8	An accepted requester shall be able to see the identity of an anonymous poster. The serializer branch exists; it takes effect once join requests are built.	High	Partial
FR-3.9	A student shall be able to open the detail view of a single goal.	High	Implemented
FR-3.10	A poster shall be able to cancel their own open goal (`CANCELLED`). Another student shall receive `403`; a goal that is no longer open shall receive `409`.	Medium	Implemented

3.2.4 Module 4: Join Requests and Approval

Build Step 4 0 of 8 implemented

Request-to-join, manual approval or auto-accept, and gender-preference enforcement.

Identifier	Requirement	Priority	Status
FR-4.1	A verified student shall be able to request to join an open, unexpired goal that is not their own. At most one active request per student per goal shall exist.	High	Planned
FR-4.2	A request shall be refused when the goal is not `OPEN`, has expired or is full, or when either student has blocked the other.	High	Planned
FR-4.3	The gender preference shall be enforced on the server: for `MALE_ONLY` or `FEMALE_ONLY` only a requester whose declared gender matches may request. A requester with gender `UNDISCLOSED` or `OTHER` shall be refused.	High	Planned
FR-4.4	The poster shall see the pending requests with the requester's identity (never anonymised) and shall be able to approve or reject each one.	High	Planned
FR-4.5	When the goal has auto-accept set, a valid request shall be accepted immediately until the headcount is met.	Medium	Planned
FR-4.6	Accepting a request shall increase the goal's accepted count. When the count equals the headcount the goal shall move to `FULL` and further requests shall be refused.	High	Planned
FR-4.7	A requester shall be able to withdraw a pending request (`WITHDRAWN`).	Medium	Planned
FR-4.8	The poster shall be notified of a new request and the requester of the decision.	Medium	Planned

3.2.5 Module 5: Lobby and Real-time Chat

Build Steps 5 and 6 0 of 7 implemented

Group lobby formed from accepted requests and the chat inside it.

Identifier	Requirement	Priority	Status
FR-5.1	When the first request for a goal is accepted a lobby shall be created in state `FORMING`, with the poster and every accepted student as members.	High	Planned
FR-5.2	The lobby shall move to `CONFIRMED` when the headcount is reached, to `COMPLETED` when the outcome is recorded, and to `CANCELLED` if the goal is cancelled while the lobby is still forming.	High	Planned
FR-5.3	Only lobby members shall be able to read or post messages in the lobby; any other student shall receive `403`.	High	Planned
FR-5.4	Messages shall be delivered in real time over WebSocket (Socket.IO). They shall be persisted, and the history shall be loaded when a member opens the lobby.	High	Planned
FR-5.5	A message shall be limited in length (proposed: 1000 characters) and shall be rendered as plain text.	Medium	Planned
FR-5.6	Messages from a student who has been blocked by the viewer shall not be shown to that viewer.	Medium	Planned
FR-5.7	A member shall be able to leave a lobby that has not yet been confirmed; the freed place shall reopen the goal.	Medium	Planned

3.2.6 Module 6: Notes and Resource Sharing

Build Step 7 0 of 3 implemented

Private one-to-one responses to a notes request, with file attachments.

Identifier	Requirement	Priority	Status
FR-6.1	A goal in the `NOTES` category shall be answered through a private one-to-one thread between the poster and each responder, not through a public lobby.	High	Planned
FR-6.2	A responder shall be able to attach files (proposed: Portable Document Format files and images, at most 10 megabytes each) that are kept in object storage and served by signed link.	Medium	Planned
FR-6.3	After receiving the material the poster shall be able to rate its quality separately from the person's reliability.	Low	Planned

3.2.7 Module 7: Trust and Safety

Build Step 8 0 of 5 implemented

Reporting, blocking, administrator review and audited unmasking.

Identifier	Requirement	Priority	Status
FR-7.1	A student shall be able to report another student or a goal with a reason; the report shall be stored for administrator review.	High	Planned
FR-7.2	A student shall be able to block and unblock another student. Blocked students shall not see each other's goals, requests or messages.	High	Planned
FR-7.3	Users shall have a role (`STUDENT` or `ADMIN`). The role column exists today; administrator-only routes are added with this module.	High	Partial
FR-7.4	An administrator shall be able to list open reports and dismiss them, warn the reported student or suspend the account.	High	Planned
FR-7.5	An administrator shall be able to unmask the poster of an anonymous goal only by giving a reason, and every unmasking shall be written to an `AuditLog`.	High	Planned

3.2.8 Module 8: Reliability, Ratings and Feedback

Build Step 9 0 of 5 implemented

Attendance confirmation, no-show flags, ratings and the reliability score.

Identifier	Requirement	Priority	Status
FR-8.1	After the goal's time, each lobby member shall confirm whether the meet-up took place. When the outcomes are recorded the lobby and the goal shall move to `COMPLETED`.	High	Planned
FR-8.2	A member shall be able to flag a no-show against a specific member. The flag shall be recorded and counted in that student's `noShowCount`.	High	Planned
FR-8.3	After completion a member shall be able to rate each other member from 1 to 5 with an optional comment, once per pair per goal.	Medium	Planned
FR-8.4	The reliability score shall start at 100, be recomputed on lifecycle events, and be shown on every profile and every feed card. Storage columns for the score and counters already exist.	High	Partial
FR-8.5	`goalsCompleted` shall be increased for every member of a lobby that completes.	Medium	Planned

3.2.9 Module 9: Matching and Priority Queue

Build Step 10 0 of 4 implemented

Rule-based goal suggestions and reliability-aware request ordering.

Identifier	Requirement	Priority	Status
FR-9.1	The feed shall be able to suggest goals ranked by shared interests, branch and categories the student has joined before.	Medium	Planned
FR-9.2	When several students request the same goal the poster's list shall be ordered by reliability and profile completeness, not only by arrival time.	Medium	Planned
FR-9.3	Hostel-block and proximity matching. Future scope: with a pilot confined to one batch, proximity had almost nothing to discriminate on.	Low	Future scope
FR-9.4	Duplicate and near-duplicate goal detection. Future scope: not part of the course project, as stated in the proposal.	Low	Future scope

3.2.10 Module 10: Metrics, Lifecycle and Operations

Build Steps 11 and 12 2 of 5 implemented

Archival sweep, evaluation metrics, continuous integration and deployment.

Identifier	Requirement	Priority	Status
FR-10.1	A student's feed shall hide expired goals at query time. A scheduled sweep shall additionally mark expired and finished goals `ARCHIVED`. The query-time part is built; the sweep is not.	Medium	Partial
FR-10.2	The system shall compute the evaluation metrics of Section 4.4 (lobby fulfilment, time-to-first-request, no-show rate, resource response rate) and show them to administrators.	Medium	Planned
FR-10.3	`GET /api/health` shall report service status, database reachability and uptime, answering `503` when the database is down.	Low	Implemented
FR-10.4	Every push shall run format, lint, type-check, test and build in continuous integration (GitHub Actions).	Medium	Implemented
FR-10.5	The frontend shall be deployed on Vercel and the application programming interface on Render or Railway, with PostgreSQL in production and SQLite in development.	Medium	Planned

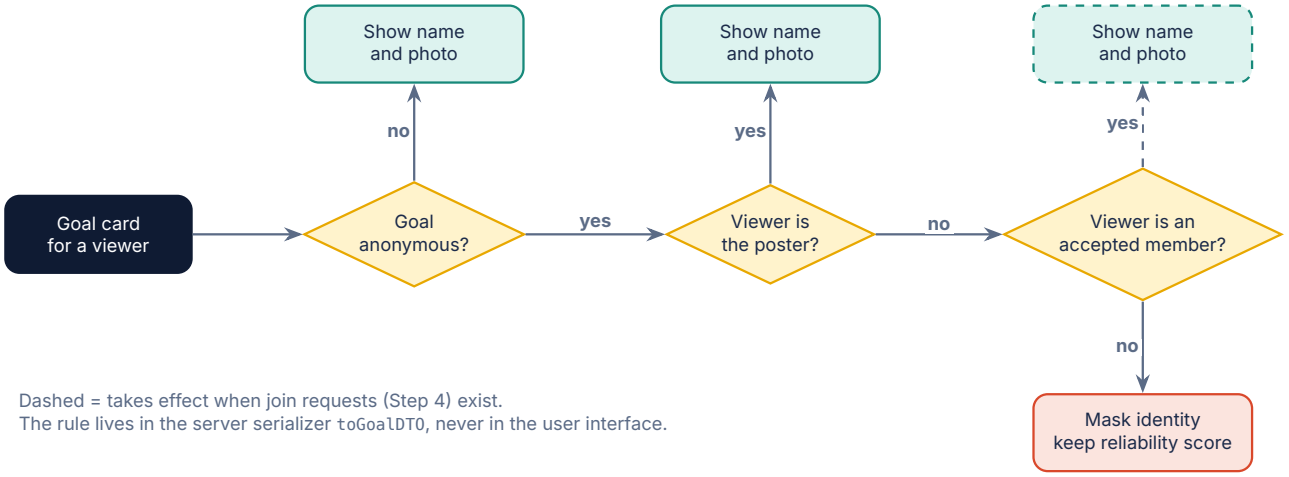


Figure 3.2. Decision applied to every goal card; the accepted-member branch becomes live with Step 4

3.3 Business Rules Behind the Requirements

3.3.1 Who can see a poster's identity

Anonymity is a pre-acceptance shield, not a permanent one. The decision is taken once, in the server-side serializer, for every goal returned to every viewer (Figure 3.2). The requester's own identity is never hidden from the poster, because the poster needs to see who is asking in order to approve.

3.3.2 Gender-preference matrix

A poster can restrict a goal to male or female requesters, for example for a late-night ride home. The rule below is the one proposed for Step 4; the preference itself is already stored and validated.

Preference	MALE	FEMALE	OTHER	UNDISCLOSED
`ANY`	allowed	allowed	allowed	allowed
`MALE_ONLY`	allowed	refused	refused	refused
`FEMALE_ONLY`	refused	allowed	refused	refused

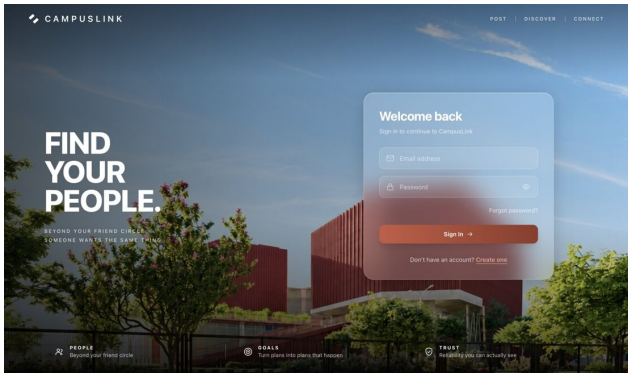
Table 3.1. Proposed enforcement of the gender preference

3.4 External Interface Requirements

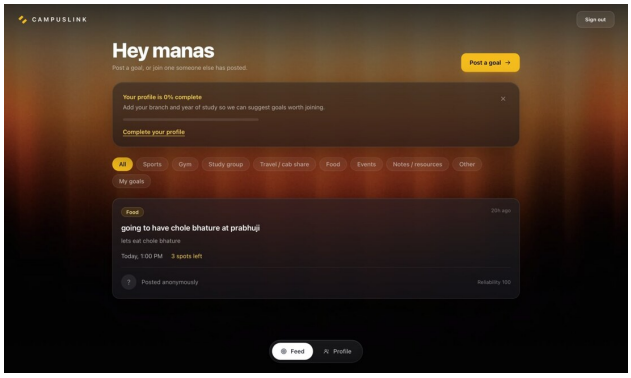
3.4.1 User interfaces

The interface is a responsive single-page application in a dark, high-contrast theme with a gold accent. A bottom navigation switches between Feed and Profile. Figure 3.3 shows three screens from the prototype.

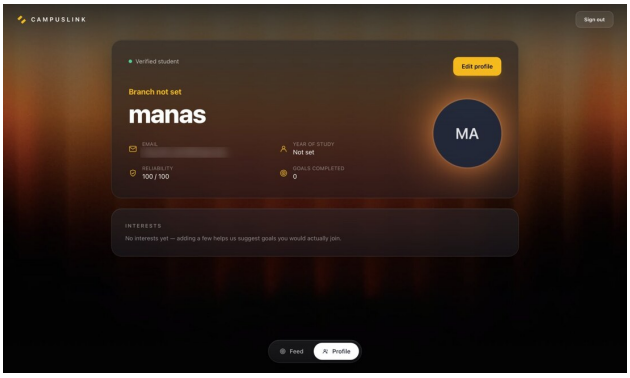
- Forms shall show validation messages next to the field and shall not clear the input on error.
- The feed card shall show category, title, time, remaining spots, poster (or "Posted anonymously") and reliability score.
- Interactive targets shall be at least 44 by 44 pixels on touch screens.



(a) Landing and sign-in



(b) Feed with category filters; one goal posted anonymously



(c) Profile with completeness and reliability

Figure 3.3. Prototype screens (e-mail address blurred)

3.4.2 Software interfaces: application programming interface

All routes are served under `/api`. Request bodies are JavaScript Object Notation, validated with Zod; the server answers with the status codes `400`, `401`, `403`, `404`, `409`, `429` and `503` where applicable. The following 16 endpoints are implemented.

Method	Path	Access	Purpose
GET	`/health`	public	Service status, database reachability, uptime.
POST	`/auth/signup`	public, limited	Create an unverified account and e-mail a one-time password.
POST	`/auth/resend-otp`	public, limited	Send a new one-time password; identical response for unknown addresses.
POST	`/auth/verify-otp`	public, limited	Verify the code, mark the account verified, return a session.
POST	`/auth/login`	public, limited	E-mail and password; returns JavaScript Object Notation Web Token and private user record.
GET	`/auth/me`	signed in	Current user from the token.
GET	`/users/me`	signed in	Own profile with completeness percentage.
PATCH	`/users/me`	signed in	Edit name, branch, year, gender, interests (strict schema).
POST	`/users/me/photo`	signed in	Upload an avatar (base64 data string; Joint Photographic Experts Group, Portable Network Graphics or WebP).
DELETE	`/users/me/photo`	signed in	Remove the avatar.
GET	`/users/:id/photo`	public	Avatar bytes, long-lived private cache.
GET	`/users/:id`	signed in	Public profile of another student (no e-mail, no role).

Method	Path	Access	Purpose
POST	/goals	signed in	Post a goal.
GET	/goals	signed in	Feed: filters category, from, to, mine; cursor, limit.
GET	/goals/:id	signed in	One goal, serialized for the viewer.
DELETE	/goals/:id	signed in, poster	Cancel an open goal.

Planned endpoints (proposed names) cover join requests (POST /goals/:id/requests, PATCH /requests/:id), lobbies and chat (GET /lobbies/:id, Socket.IO), reports and blocks (POST /reports, PUT /blocks/:userId), admin unmasking and metrics (/admin/*), and ratings (POST /ratings).

3.4.3 Hardware, communication and external-service interfaces

- **Hardware:** none beyond a device with a browser. The application programming interface needs no special hardware.
- **Communication:** Hypertext Transfer Protocol Secure between browser and application programming interface in production; JavaScript Object Notation over representational state transfer; WebSocket for chat (planned). Cross-origin resource sharing is limited to the front-end origin.
- **Simple Mail Transfer Protocol:** nodemailer sends the one-time password e-mail through a configured relay (port 465 with Transport Layer Security, or 587 with STARTTLS).
- **Object storage (planned):** Cloudinary or Firebase for notes attachments, reached from the server only.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

CampusLink is sized for a pilot of 15 to 20 students, so the targets below are modest. They are *proposed* and shall be measured in the pilot.

- **Application programming interface latency:** the 95th percentile of feed, goal-detail and profile requests shall stay below 300 milliseconds on the free-tier host.
- **Page load:** the single-page application shall become interactive in under 3 seconds on a typical campus Wi-Fi or 4G connection.
- **Feed query:** the feed shall be cursor-paginated (default 20, maximum 50 per page) and served by indexes on status with creation time, status with category, and date; query cost shall not grow with the number of old goals.
- **Chat delivery (planned):** a message shall reach every online lobby member within 1 second.
- **Payload limits:** JavaScript Object Notation bodies are limited to 1 megabyte; avatars to 250 kilobytes after resizing to 320 pixels.

4.2 Security Requirements

- **Credential storage:** passwords are stored only as salted scrypt hashes; one-time password codes only as hashes, single-use, 10-minute expiry, 5 attempts.
- **Injection:** all database access goes through Prisma's parameterised queries; request bodies are validated with Zod before use.
- **Cross-site scripting:** React escapes output, and the front end contains no dangerouslySetInnerHTML. Chat text shall be rendered as plain text.

- **Access control:** every route except health, auth and the avatar image needs a valid Bearer token; only the poster may cancel a goal; administrator routes require `role = ADMIN`.
- **Mass assignment:** the profile update schema is strict and the service uses an explicit allow-list, so a student cannot set `role`, `isVerified` or `reliabilityScore`.
- **Enumeration:** resend-one-time-password and login give the same response for unknown and known addresses.
- **Abuse control:** authentication endpoints are limited to 20 requests per client per 15 minutes; `helmet` sets secure headers.
- **Anonymity:** enforced in the server serializer, never in the user interface, so a hidden identity cannot be recovered from the application programming interface response.
- **Auditability (planned):** every administrator unmasking is written to an `AuditLog` with the reason.

Known security limitations

(1) The session token is kept in `localStorage`, so a cross-site-scripting flaw would expose it, and logout only discards it on the client. (2) E-mail aliases such as `name+tag@thapar.edu` are accepted as separate accounts, which lets a student re-register to shed a bad reliability score; the fix is a canonical-e-mail unique column and is deferred for the pilot. (3) Avatar images are served from an unauthenticated uniform resource locator with a long private cache so that browsers can reuse them. They are not secret, but they are reachable by anyone who knows a user id.

4.3 Software Quality Attributes

- **Usability and responsiveness:** fluid layouts for phone and desktop; touch targets of at least 44 by 44 pixels; clear, specific error messages.
- **Reliability:** at least 99% availability during evaluation sessions; the client shall handle network errors without losing typed input.
- **Maintainability:** a monorepo with a layered back end (route, controller, service, serializer), one shared package for schemas and enumerations, and strict TypeScript. Continuous integration runs format, lint, type-check, test and build on every push.
- **Testability:** more than 90 automated tests cover auth, profile, goals, health, one-time password, passwords, e-mail templates and profile completeness.
- **Portability:** the same code runs on SQLite in development and PostgreSQL in production through Prisma.
- **Scalability:** the application programming interface is stateless and the feed is paginated and indexed, so the design does not prevent horizontal scaling, even though a pilot does not need it.

4.4 Business Rules and Evaluation Metrics

4.4.1 Business rules in force

Rule	Value
E-mail domain	`thapar.edu`, stored lower-cased
Password	8 to 128 characters
One-time password	6 digits; valid 10 minutes; 5 attempts; single use
Session	JavaScript Object Notation Web Token, default lifetime 7 days (configurable)
Goal title and description	4 to 80 and 10 to 600 characters
Headcount	1 to 20, excluding the poster
Goal expiry	Dated goal: at its date and time. Undated goal: 48 hours after posting
Feed page	Default 20, maximum 50
Interests	Up to 10 tags of up to 24 characters
Avatar	Joint Photographic Experts Group, Portable Network Graphics or WebP; 320 pixels; at most 250 kilobytes
Auth rate limit	20 requests per 15 minutes per client

Table 4.1. Constants fixed by the implementation

4.4.2 Reliability score

Every student starts at 100. The score is stored on the user and recomputed whenever a goal's outcome is recorded, because it is printed on every feed card and must not need a query per card. The following formula is *proposed* for Step 9:

$$R = \max\left(0,\ 100 - 10\,n_{\text{no-show}} - 5\,n_{\text{upheld}}\right) \tag{4.1}$$

where $n_{\text{no-show}}$ is the number of confirmed no-shows and n_{upheld} the number of reports upheld by an administrator. Ratings are shown next to the score and do not enter it.

4.4.3 Evaluation metrics

The primary metric tells whether the core loop works. The secondary metrics test the discovery and the trust features. Table 4.2 gives each metric, its formula, what it shows and where its data comes from.

Table 4.2. Evaluation metrics measured in the pilot

Metric	Type	Formula	What it shows and where the data comes from
Lobby fulfilment rate	Primary	$\dfrac{\text{goals reaching headcount in time}}{\text{goals posted}} \times 100\%$	Whether the core loop works. Data: goal headcount, accepted count and deadline.
Time to first request	Secondary	$T_{\text{first}} = t_{\text{first request}} - t_{\text{posted}}$	How quickly a new goal is noticed on the feed. Data: goal posting time and first request time.
No-show rate	Secondary	$\dfrac{\text{confirmed lobbies with a no-show}}{\text{confirmed lobbies}} \times 100\%$	Whether reliability scores and no-show tracking improve attendance. Data: lobby outcome records.
Resource response rate	Secondary	$\dfrac{\text{notes requests answered}}{\text{notes requests posted}} \times 100\%$	Whether notes and resource requests get answered. Data: notes requests and private responses.

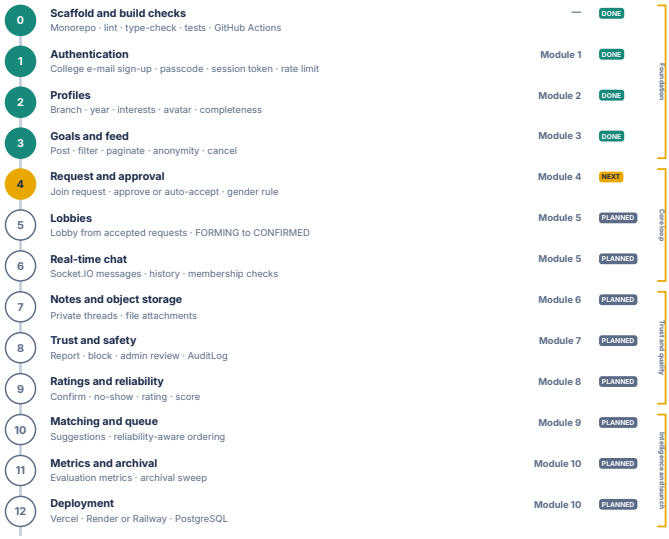


Figure 5.1. Build order of CampusLink; steps 0 to 3 are done, step 4 is next

Metric	Type	Formula	What it shows and where the data comes from
Perceived safety	Secondary	Mean of the 1 to 5 survey scores	Whether anonymous posting and the gender preference feel safe to students. Data: one survey question asked after use.

The deadline of a goal is its own date and time, or 48 hours after posting for an undated goal.

5 Other Requirements

5.1 Future System Evolution

- **Push notifications** on lobby activity.
- **Streaks and badges** for reliable students, and a **trending goals** section.
- **Canonical e-mail column** to close the alias loophole if the user base ever grows beyond a pilot.
- **Object storage for avatars,** moving them out of the database behind the same interface used for notes.
- **Password reset** through an e-mailed one-time password (FR-1.12).
- **Hostel-block and proximity matching** (FR-9.3), once the user base is larger than one batch.
- **Duplicate and near-duplicate goal detection** (FR-9.4).

5.2 Implementation Roadmap

The build order is fixed by dependency. Steps 0 to 3 are complete; Step 4 is next. Figure 5.1 lists all thirteen steps with the requirement module each one delivers.

5.3 Risks

Risk	Effect	Mitigation
Cold feed in the pilot	No goals, so request and lobby flows cannot be tested.	Seed a few goals alongside real ones.
Real-time chat on free tier	Sleeping instances delay the first WebSocket message.	Fall back to polling; keep the application programming interface stateless.
Alias accounts	A student sheds a bad score by re-registering.	Accepted for the pilot; canonical e-mail later.
Gender-rule edge cases	Students with undisclosed or other gender are refused on a restricted goal.	Document the rule; revisit after the pilot.
Mail delivery delay	Sign-up stalls while the passcode is awaited.	Resend endpoint and a clear message.

Table 5.1. Main project risks

6 Assumptions and Open Points

These items were decided by the authors while writing this document and are collected here so that they can be confirmed or corrected.

1. **Gender rule.** A requester whose gender is `UNDISCLOSED` or `OTHER` is refused on a `MALE_ONLY` or `FEMALE_ONLY` goal (Table 3.1).
2. **Performance targets** in Section 4.1 (300 milliseconds, 3 seconds, 1 second) are proposed, not measured.
3. **Planned endpoint names** in Section 3.4 are proposals.
4. **Reliability formula** in Section 4.4 is proposed; the proposal only requires that no-shows are factored in.
5. **Limits** of 1000 characters per chat message and 10 megabytes per attachment are proposed.
6. **`CLOSED` goal status** exists in the enumeration but has no defined transition.
7. **One-time password at login.** The proposal could be read as asking for a one-time password at every login. The implementation verifies once at sign-up, and this document follows the implementation.
8. **Lobby cancellation.** A lobby can be cancelled only while forming, because the implemented rule lets a poster cancel only an `OPEN` goal.